AIVS: Anonymous Identification Verification System

Based on Ring Signatures with Isolated Smart Card Hardware

Dinh Anh Khoa Nguyen - 245849840 - nguy9840@mylaurier.ca
Jiangtao Wang - 245823800 - Wang2370@mylaurier.ca
Leila Chen - 245850620 - chen5062@mylaurier.ca
Yuyang Wu - 245817310 - wuxx7310@mylaurier.ca

April 2026

Abstract

This report outlines the engineering implementation of a privacy-preserving messaging platform that decouples message integrity from identity revelation. Leveraging Yubikey hardware PRF and nested Ring Signatures, the system allows users to send undeniable, tamper-proof messages anonymously, with the exclusive capability to retroactively prove authorship. The project adopts an Agile methodology, utilizing React/TypeScript for the frontend and Node.js for the backend.

Contents

1 Introduction **3**

2 Problem Motivation **3**
2.1 Problem Statement . . . 3
2.2 Objectives . . . 3
2.3 Scope and Limitations . . . 4
2.4 Report Structure . . . 4

3 Background and Related Work **4**
3.1 Anonymous Communication Systems . . . 4
3.2 Ring Signatures — Theory and Prior Work . . . 4
3.3 Hardware Security Tokens and WebAuthn PRF . . . 4
3.4 Gap Analysis . . . 4

4 System Design **4**
4.1 Architecture Overview . . . 4
4.2 Security Model and Threat Model . . . 4
4.3 Cryptographic Construction . . . 4
4.4 Component Diagram . . . 5
4.5 Data Flow Diagram . . . 5
4.6 Database Schema . . . 5

5 Implementation **5**
5.1 Technology Stack . . . 5
5.2 Cryptography Module . . . 6
5.3 YubiKey Integration via WebAuthn PRF . . . 7
5.4 Backend API . . . 7
5.5 Frontend UI . . . 7
5.6 End-to-End Flow: Alice → Bob → Reveal . . . 9

6 Evaluation **10**
6.1 Functional Testing . . . 10
6.2 Security Analysis . . . 10
6.3 Performance Analysis . . . 10
6.4 Limitations and Edge Cases . . . 10

7 Project Management **10**
7.1 Agile Methodology and Sprint Summary . . . 10
7.2 Milestone Outcomes vs. Planned Goals . . . 12
7.3 Team Contributions . . . 12

8 Conclusion and Future Work **12**
8.1 Summary of Achievements . . . 12
8.2 Future Directions . . . 13

1 Introduction

2 Problem Motivation

2.1 Problem Statement

In modern digital communication, there is always a trade-off between authenticity and anonymity. As more and more users are concerned about surveillance and data breaches, a system that can protect user identity becomes more and more important. In the meanwhile, verifiability and accountability of message are still crucial factors for the trustworthiness of the digital communication.

However, most of the existing solutions tend to compromise one side over the other instead of addressing both at the same time.

OpenPGP or S/MIME are examples of Public Key Infrastructure (PKI), where digital signatures are tied to a long-term identity. This mechanism ensures the verifiability of the original message and non-repudiation, but it also limits anonymity as every signed message can be traced back to the user. This demonstrates a typical trade-off between authenticity and anonymity in the real-world scenario.

In terms of anonymous communication systems, those Tor–based systems are designed to hide user identity. These systems focus effectively on a privacy perspective, but they also introduce the difficulty of verification for the authorized message sender as there is no sign of identity. This increases the risk of impersonation and Sybil attacks, and it also prevents users from later proving authorship when needed. This limitation is essential to be addressed under scenarios like whistleblowing or dispute resolution, where both anonymity and credibility are required.

Another problem related to key management is that private keys are still handled in software in many cryptographic schemes, including those designed for anonymity like ring signatures. This deficit significantly compromises the system and makes it exposed to common attack vectors such as memory inspection or malware.

Overall, these existing schemas suggest the need for a solution that can fully ensure both privacy and accountability. To address this gap, a mechanism that allows messages to remain unlinkable by default, while still giving the original sender the ability to reveal authorship later in a controlled and verifiable way, without relying on a third party, is introduced in this project.

2.2 Objectives

The main goal of this project is to design a system that separates the act of signing a message from the act of revealing identity, while still maintaining both security and practical usability.

To guide the design, the following objectives are defined:

- Objective A: Hardware-Isolated Key Derivation
  The system uses the WebAuthn PRF extension to derive one-time signing keys (OTK) with the help of a hardware security device. By doing so, sensitive key material does not leave the hardware boundary, which helps reduce the risk of key exposure even if the host environment is not fully trusted.

- Objective B: Nested Ring Signature Construction
  A two-layer ring signature scheme is introduced using a 1-of-N structure (with $N = 8$). This allows a message to be verified as valid while keeping the actual signer indistinguishable from other members of the group.

- Objective C: Selective Disclosure Mechanism
  The system supports a challenge-response process that allows the original sender to prove authorship when necessary. This is achieved through an encrypted proof (Sig_{inner}), which can only be accessed using a hardware-derived secret. As a result, identity disclosure remains entirely under the control of the sender.

- Objective D: Practical Performance
  Since the system is intended for use in a browser environment, performance is also taken into account. In the meanwhile, a non-delay user interaction is guaranteed by keeping cryptographic operations efficient.

2.3 Scope and Limitations

2.4 Report Structure

3 Background and Related Work

3.1 Anonymous Communication Systems

3.2 Ring Signatures — Theory and Prior Work

3.3 Hardware Security Tokens and WebAuthn PRF

3.4 Gap Analysis

4 System Design

4.1 Architecture Overview

The system follows a **Thick Client, Thin Server** architecture. The server acts purely as a bulletin board and relay, possessing zero knowledge of user identities or private keys, and is explicitly excluded from the trusted computing base.

4.2 Security Model and Threat Model

- **Key Derivation:** The OTK is derived with the participation of the Yubikey, ensuring it cannot be predicted or extracted.

- **Anonymity Set:** 1 real signer + 7 random decoys = 8 total keys. Probability of guessing the signer is 1/8 per message (statistically negligible for identification).

- **Forward Secrecy:** Compromise of the Yubikey allows decryption of past Proofs but does not reveal the identity of past messages *unless* the attacker actively performs the decryption and linking.

4.3 Cryptographic Construction

1. **Setup:** Generate One-Time Key (OTK) via Yubikey. Generate 7 Fake Keys ($FK_{1...7}$).

2. **Inner Layer (The Proof):**

$$Sig_{inner} = RingSign(Message, \{OTK, FK_{1...7}\})$$

$$Key_{proof} = KDF(PRF_{Yubikey}(Hash(OTK)))$$

$$Proof = Encrypt_{AES-GCM}(Sig_{inner}, Key_{proof})$$

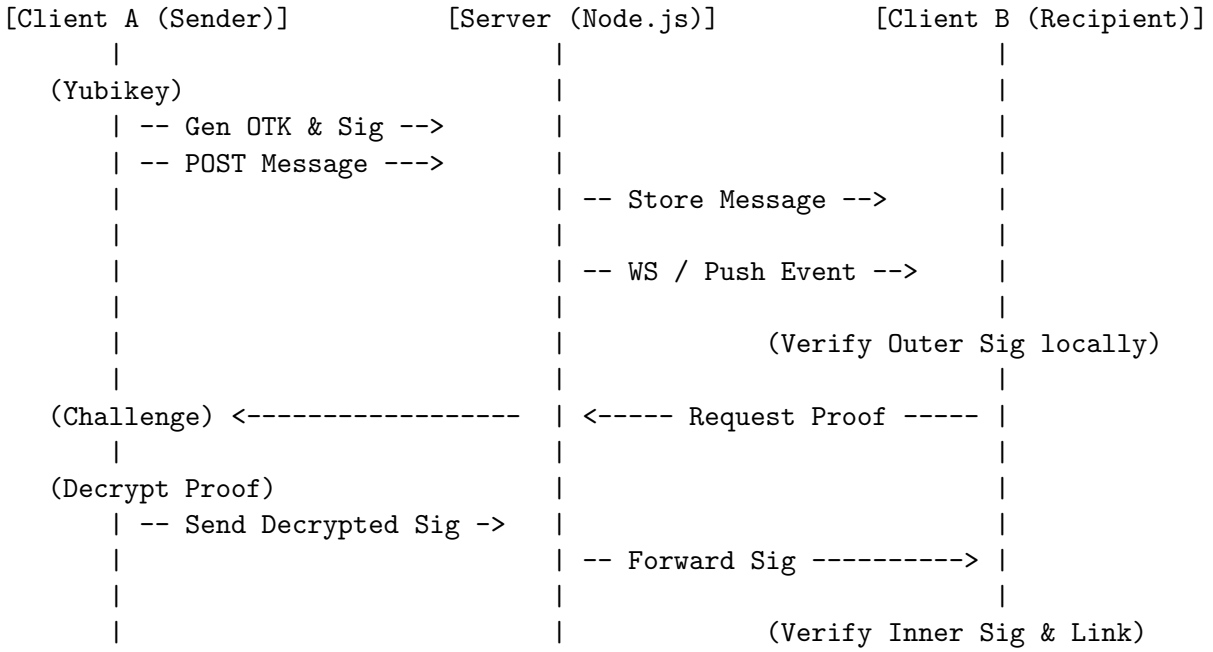
where the hash of the OTK is used as a salt to bind the proof key to the one-time signing key.

3. **Outer Layer (The Transmission):**

$$Sig_{outer} = RingSign(Message||Hash(Sig_{inner}), \{OTK, FK_{1...7}\})$$

4. **Verification:** Upon decryption, the Linkable Tag of Sig_{inner} must equal the Linkable Tag of Sig_{outer}.

4.4 Component Diagram



4.5 Data Flow Diagram

1. **Registration:** None required for the messaging protocol itself (stateless). Optional username registration maps public handles to WebAuthn Credential IDs for routing.

2. **Signing:** All cryptographic operations occur in the browser memory. The Yubikey is accessed via `navigator.credentials.get({prf: ... })`.

4.6 Database Schema

5 Implementation

5.1 Technology Stack

- **Frontend:** React, TypeScript.

- **Cryptography (Client):** WebAuthn API (PRF extension), Elliptic Curve Library (e.g., `elliptic` or WASM-based Rust bindings) for Ring Signatures.
- **Backend:** Node.js (Express/NestJS).
- **Database:** PostgreSQL (Stores only blobs: Message, Public Keys Ring, Signature, Encrypted Proof).

5.2 Cryptography Module

This module implements the core cryptographic operations. The core functions of this module are implements of a LRS based on Ristretto255 and Ed25519 curves. We can divided the module into four main components: core cryptographic implementation, key management, serialization tools and utility Functions.

Core Cryptographic Implementation The core functionality of the part revolves around LRS, which allows a user to sign a message anonymously on behalf of a group (ring) while providing a way to "link" signatures if the same member signs multiple times.

- **Data Structures**: It defines `LRSSignature` and `LRSSignatureRaw` interfaces to store signature components: `c0` (initial hash), `r` (responses), `I` (linkability tag/key image), and `ring` (public keys).
- **Signature Generation**:
  - `generateLRS`: Generates a signature given a message, a set of public keys, the signer's index, and the signer's private key.
  - `generateLRSWithoutFakeKeys`: A helper that automatically generates random "fake" keys to fill the ring, simplifying the signing process for a single user.
- **Verification**:
  - `verifyLRS`: Validates the signature by reconstructing the hash chain across the ring and checking if it matches the provided `c0`.

Key Management This part provides methods for managing cryptographic keys using the Ristretto255 curve:

- `generateKeyPair`: Creates a public/private key pair, optionally from a provided seed for deterministic generation.
- `validatePublicKey`: Verifies that a specific public key corresponds to a given private key.

Serialization Tools To facilitate data transmission and storage, the part includes utilities to convert signature objects to and from byte formats.

- `LRSToBytes`: Serializes an `LRSSignature` object into a `Uint8Array`.
- `bytesToLRS / bytesToRawLRS`: Deserializes a byte array back into a structured signature object.

Utility Functions

- **Hashing**: Includes `sha256` for standard data hashing, `hashToScalar` for curve-specific scalar generation, and `H_p` (Hash-to-Curve) for mapping data to a point on the Ristretto255 curve.
- **Data Conversion**: Exports standard utilities like `hexToBytes` and `bytesToHex` for handling hexadecimal strings.
- **Comparison**: `uint8ArrayEquals` for comparing two byte arrays for equality.

Conclusion In conclusion, the cryptography module is designed to be efficient and secure, ensuring that all operations are performed locally on the client side with the assistance of the Yubikey, while the server remains a passive relay that handles only opaque data blobs.

5.3 YubiKey Integration via WebAuthn PRF

This component handles the interaction with the YubiKey hardware token using the WebAuthn API, specifically leveraging the PRF extension to derive one-time keys for signing and encryption. This is the main component that ensures the security of the key material by keeping it within the hardware boundary. For its implementation, it can be divided into two different parts:

WebAuthn and Hardware Key Derivation The system utilizes the **WebAuthn PRF (Pseudo-Random Function) extension** to securely derive cryptographic keys from the user's hardware authenticator (e.g., TouchID, FaceID, or YubiKey) rather than storing private keys in plain text.

- `getCredential`: Triggers the `navigator.credentials.get` API with PRF extensions enabled to fetch hardware-backed credentials.
- `generateKeyWithPRF`: Uses the derived key from the hardware as a seed to generate deterministic Ristretto255 asymmetric key pairs.

Cryptographic MessagePack Processing This is the core privacy logic that implements Linkable Ring Signatures (LRS) to ensure anonymity and verifiability.

- `getMessagePack`: Orchestrates the creation of a secure message pack. It combines the payload with a timestamp/nonce, generates an *internal* LRS, encrypts it via AES-GCM, and wraps it in an *external* LRS.
- `ValidateMessagePack`: Checks metadata versioning and verifies the external ring signature to ensure message integrity.
- `getProofForReceiver`: Enables the receiver to attempt decryption of the AES-GCM ciphertext using their own derived keys to verify the sender.

5.4 Backend API

5.5 Frontend UI

The frontend project is structured as a Single-Page Application (SPA). The overall architecture can be summarized as: **Single-Page Entry + Authentication Gateway + Signing Workbench**

+ Data Request Utility Layer. In terms of implementation style, it follows a React functional layered architecture, with the core business logic centered around WebAuthn authentication and cryptographic signing workflows.

Application Entry Layer

- `index.html`: Provides the `root` container and loads `/src/main.tsx`.
- `src/main.tsx`: Creates the React Root, injects `StrictMode`, and mounts the application.

Application Shell Layer

- `src/App.tsx`: The application shell component; currently lightweight, primarily responsible for rendering the login wrapper component.
- `src/App.css`: Global base styles (root container and heading styles).

Authentication and Session Layer

- `src/login.tsx`: The primary business entry point, responsible for fetching registration/login challenges, invoking WebAuthn, managing session tokens, and switching the UI post-login.
- Supports Message Pack file imports, serving as the bridge between the authentication domain and the business domain.

Business Feature Layer (Signing and Verification)

- `src/Sign.tsx`: An aggregation layer for the signing workbench, using a collapse panel to organize features:
  - Create new signed messages
  - Message Pack list
  - Received/Sent verification request lists
- `src/Message.tsx`: List details and operations layer, responsible for:
  - Message Pack detail display and local validation
  - File export
  - Ownership transfer
  - Sending identity confirmation requests
  - Request verification/rejection workflows

Domain and Infrastructure Layer

- `src/utils.ts`: The core utility layer, handling API communication, cryptographic process encapsulation, authentication header assembly, and type definitions.
- This file also provides Message Pack construction and validation logic, serving as a critical dependency for upper-layer components.

Reserved Layer

- `src/constants.ts`: Currently empty; can serve as a unified entry point for constants in the future.
- `src/assets/`: Currently stores static resources (e.g., `react.svg`).

Rendering Flow This application rendered in this flow:
`index.html` → `main.tsx` → `App.tsx` → `login.tsx` → `Sign.tsx` → `Message.tsx`

State Flow The project relies primarily on React local state (`useState` or `useEffect` or `useCallback`); no global state management library is introduced. Session state is persisted via `sessionStorage`, and user state is restored via session APIs after a refresh.

Data Flow UI components call functions in `src/utils.ts`, and the utility layer performs unified access to the backend `/api`. In the development environment, Vite proxies and forwards requests to local backend services.

Dependency Boundaries

- UI and Interaction: React + Ant Design
- Shared Types and Cryptography: `@repo/shared`
- Build and Development: Vite + TypeScript + ESLint

Strengths

- Clear Layering: Distinct boundaries between page orchestration and infrastructure capabilities.
- Comprehensive Typing: Request/response types are centrally defined in the utility layer.
- Complete Business Loop: Covers core processes including login, signing, requesting, verification, and import/export.

5.6 End-to-End Flow: Alice → Bob → Reveal

The system involves three primary roles: the Sender, the Recipient, and a Backend Server, with a YubiKey hardware token participating in cryptographic operations. Both the Sender and the Recipient interact with the system through a web browser, while the backend server functions solely as a message relay and storage service.

All cryptographic operations are performed locally on the client side with the assistance of the YubiKey. The server does not generate, store, or process private keys, and only handles opaque data blobs, including messages, signatures, and encrypted proofs. The YubiKey is used to provide hardware-bound PRF outputs that participate in one-time key derivation and proof encryption, without exposing secret material to the browser or operating system.

The system does not maintain a global signing key registry. Each message is associated with a freshly derived one-time key, and no signing keys are reused across messages.

During message transmission, the Sender composes a message using a browser-based client with a connected YubiKey. A one-time key is derived with the participation of the YubiKey and used,

together with a fixed set of randomly generated decoy keys, to construct a ring signature over the message. To support future authorship verification, an additional inner ring signature is generated and encrypted as a proof using a symmetric key derived from the YubiKey PRF output. An outer ring signature is then computed over the message concatenated with the hash of the encrypted proof. The message, outer signature, and encrypted proof are transmitted to the Recipient via the backend server.

Upon receipt, the Recipient verifies the outer ring signature to confirm message integrity and authenticity, while the Sender's identity remains hidden within the anonymity set. The encrypted proof is not required for this initial verification.

If identity verification is later requested, the Recipient sends the original message, signature, and encrypted proof to the claimed Sender. Using the same YubiKey, the Sender derives the corresponding decryption key and decrypts the proof to reveal the inner ring signature. The decrypted signature is returned to the Recipient, who verifies its validity and linkability against the original message and signature to confirm authorship.

6 Evaluation

6.1 Functional Testing

6.2 Security Analysis

6.3 Performance Analysis

6.4 Limitations and Edge Cases

7 Project Management

7.1 Agile Methodology and Sprint Summary

Sprint 0: Feasibility & PoC (2 Weeks)

- **Goal:** Validate WebAuthn and Yubikey PRF behavior in the Chrome/Firefox browser.
- **Tasks:**
  - Test WebAuthn registration and authentication process in Chrome/Firefox browser
  - Validate PRF output by using identical salts
  - Derive AES keys from PRF output
- **Deliverable:** A minimal browser-based script that inputs a salt and outputs a consistent AES key using a Yubikey.

Milestone 1: Core Cryptography Module (2 weeks)

- **Goal:** Implement Linkable Ring Signatures in TypeScript/WASM.
- **Tasks:**
  - Implement a one-time key from WebAuthn
  - Implement linkable ring signatures (1-of-8) in Typescript/WASM
  - Implement linkability tag generation and verification

  - Unit tests for signing, verification, and tag

- **Deliverable:** Unit tests proving 1-of-8 signing and verification, and tag linking.

Milestone 2: Backend & Basic UI (2 weeks)

- **Goal:** Message transport and basic user interaction.
- **Tasks:**
  - Implement Node.js backend for storing and retrieving message
  - Design message, signature, and proof data format
  - Design and implement a React frontend user interface for composing and viewing messages
  - Integrate WebAuthn for user login
- **Deliverable:** Node.js API to store/retrieve message blobs. React UI to compose messages.

Milestone 3: Integration & The "Reveal" Flow (2 weeks)

- **Goal:** Connect crypto with UI and demonstrate end-to-end flow.
- **Tasks:**
  - Integrate ring signature into the message
  - Integrate proof encryption and decryption using YubiKey PRF
  - Implement complete end-to-end flow from proof request, decryption, and verification
  - Validate and test the flow
- **Deliverable:** End-to-End flow: Alice sends anonymous msg → Bob requests proof → Alice reveals → Bob verifies.

Milestone 4: Evaluation and Documentation (1 week)

- **Goal:** Testing the system and prepare for evaluation and document.
- **Tasks:**
  - Bug fixes and handle edge-cases if persist
  - Security, threat, and performance review
  - Documentation
- **Deliverable:** A stable prototype with documentation.

7.2 Milestone Outcomes vs. Planned Goals

7.3 Team Contributions

8 Conclusion and Future Work

8.1 Summary of Achievements

The objectives achieved by the project so far can be summarized into the following six levels:

1. **Established a Comprehensive Full-stack Monorepo Architecture**
   - Organized a three-tier structure (frontend, backend, and shared libraries) using pnpm workspaces.
   - Unified use of TypeScript across both frontend and backend, with reusable shared types and utility functions.
   - Supported parallel development and unified builds (root directory scripts have been configured).
2. **Implemented the Main WebAuthn-based Authentication Workflow**
   - Backend provides endpoints for registration challenges, registration completion, login challenges, login completion, and session queries.
   - Frontend completed the implementation of registration and login flows using the browser's WebAuthn API.
   - Implemented subsequent interface authentication via session tokens (Header or Bearer).
   - Supported session persistence and recovery (session can be retrieved after page refresh).
3. **Implemented the Business Closed-loop for Message Packs as Transferable Assets**
   - Supported the creation of Message Packs (Content + Memo + CredentialId).
   - Supported querying by ID and by username lists.
   - Supported ownership transfer (change of owner).
   - Data models and indexes are persisted in the database, providing basic retrieval performance.
4. **Implemented Identity Validation Request Workflows**
   - Supported initiating requests (requester → responder).
   - Supported querying "Requests I Sent" and "Requests I Received."
   - Responders can execute respond/refuse actions;
   - Requesters can write final results: Passed/Failed/Error.
   - Backend implemented state machine constraints and protection against illegal state transitions.
5. **Implemented Cryptographic Message Signing and Verification (Shared Library + Frontend)**
   - Implemented signing, serialization, and verification based on ristretto255 / linkable ring signature (LRS).

- Used PRF for key derivation combined with AES-GCM for encapsulating internal proofs.
- Frontend supports generating Message Packs, verifying signatures, importing/exporting Message Packs, and triggering the verification process.

6. **Implemented a Deployable and Runnable Local Environment**
   - Orchestrated PostgreSQL, API, and Nginx using docker-compose.
   - Nginx supports HTTP-to-HTTPS redirection and reverse proxying for /api.
   - init.sql includes core table structures and indexes for initialization capabilities.
   - API integrates Swagger documentation (/docs) and healthz/readyz monitoring endpoints.

8.2 Future Directions

We would contribute to the project in the future by implementing the following features:

1. **UI Enhancements and User Experience Improvements**
   - Implement a more intuitive and visually appealing user interface for message composition and verification.
   - Add real-time notifications for incoming messages and verification requests.
   - Provide better error handling and user feedback during cryptographic operations.
2. **Bug catching and Fixing**
   - Conduct thorough testing to identify and fix any bugs or edge cases that may arise during usage.
   - Implement additional unit and integration tests to ensure the robustness of the system.
3. **Performance Optimization**
   - Optimize cryptographic operations to reduce latency, especially for larger messages or higher volumes of messages.
   - Explore caching strategies for frequently used keys or signatures to improve performance.

Appendix A: Code Repository

Appendix B: Database

Appendix C: ReadMe